

机器学习实验指导手册

支持向量机（SVM）模型

一、实验目的

- 掌握支持向量机（SVM）的数学原理与分类思想
 - 理解间隔最大化、核函数、软间隔等核心概念。
- 熟悉 SVM 的对偶问题与优化方法
 - 了解拉格朗日乘数法在 SVM 中的应用。
- 学会使用 **scikit-learn** 工具包实现 SVM 模型
 - 掌握数据预处理、模型训练与评估的完整流程。
- 分析不同核函数与超参数对模型性能的影响
 - 通过实验对比线性核、多项式核和高斯核的适用场景。

二、实验原理

1. 数学模型

线性可分 SVM:

目标函数为最大化分类间隔:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$
$$\text{s.t. } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \forall i$$

软间隔 SVM（处理非线性可分数据）:

引入松弛变量 ξ_i 和正则化参数 C :

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$
$$\text{s.t. } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

2. 核技巧:

将输入空间映射到高维特征空间，核函数定义为：

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

常用核函数：

- 线性核： $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- 多项式核： $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$
- 高斯核（RBF）： $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

三、实验步骤

1. 数据准备

使用 `sklearn` 内置的乳腺癌数据集：

```
```python
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer() X, y = data.data, data.target
```

### 2. 数据预处理

标准化数据并划分训练集与测试集：

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

### 3. 模型训练与评估

使用线性核函数：

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
model = SVC(kernel='linear', C=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"准确率: {accuracy_score(y_test, y_pred):.2f}")
print("混淆矩阵:", confusion_matrix(y_test, y_pred))
```

使用高斯核函数：

```
model_rbf = SVC(kernel='rbf', gamma=0.1, C=1.0)
model_rbf.fit(X_train, y_train)
y_pred_rbf = model_rbf.predict(X_test)
```

#### 4. 可视化支持向量

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

#### 降维可视化

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_train, cmap='coolwarm')
plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=100,
 facecolors='none', edgecolors='k', label='Support Vectors')
plt.legend() plt.show()
```

## 四、实验要求

### 必做任务

样本数量：569  
标准化后样本数量：569

#### 1. 实现不同核函数的对比

- 分别使用线性核、多项式核（设置 `degree=3`）、高斯核训练模型，记录准确率与混淆矩阵。

#### 2. 分析正则化参数 **C** 的影响

- 设置 `C=[0.1, 1, 10]`，观察模型在训练集和测试集上的性能变化。

测试集结果：

正则化参数  $c=0.1$ :

测试集准确率：0.9737

测试集混淆矩阵：

```
[[41 2]
 [1 70]]
```

正则化参数  $c=1$ :

测试集准确率：0.9561

测试集混淆矩阵：

```
[[41 2]
 [3 68]]
```

正则化参数  $c=10$ :

测试集准确率：0.9649

测试集混淆矩阵：

```
[[42 1]
 [3 68]]
```

训练集结果：

正则化参数  $c=0.1$ :

训练集准确率：0.9868

训练集混淆矩阵：

```
[[165 4]
 [2 284]]
```

正则化参数  $c=1$ :

训练集准确率：0.9868

训练集混淆矩阵：

```
[[166 3]
 [3 283]]
```

正则化参数  $c=10$ :

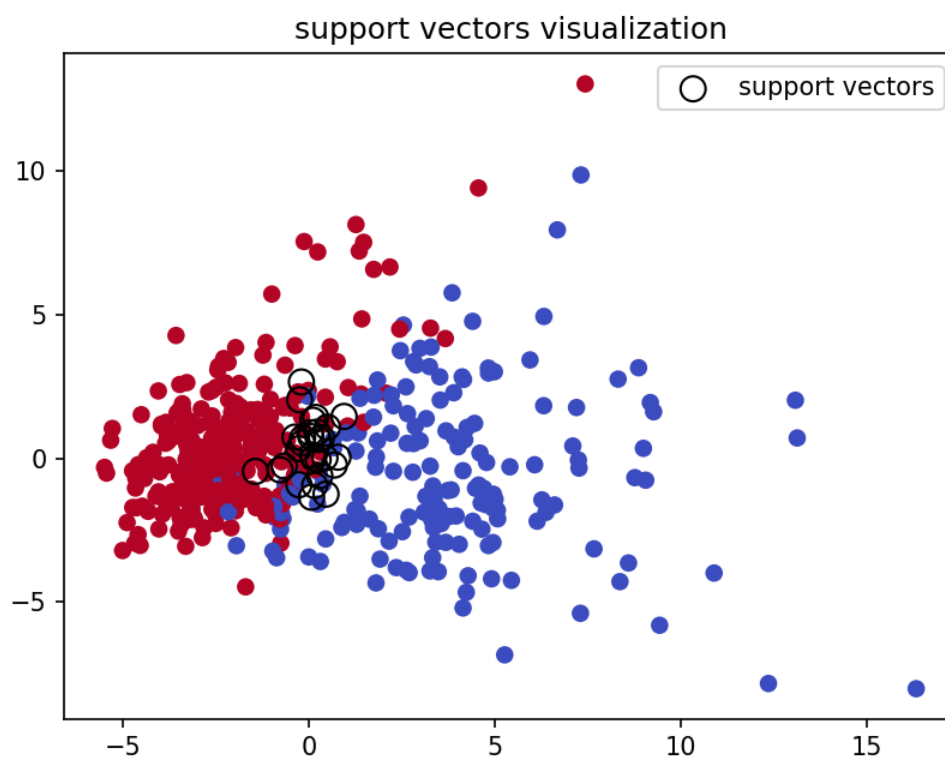
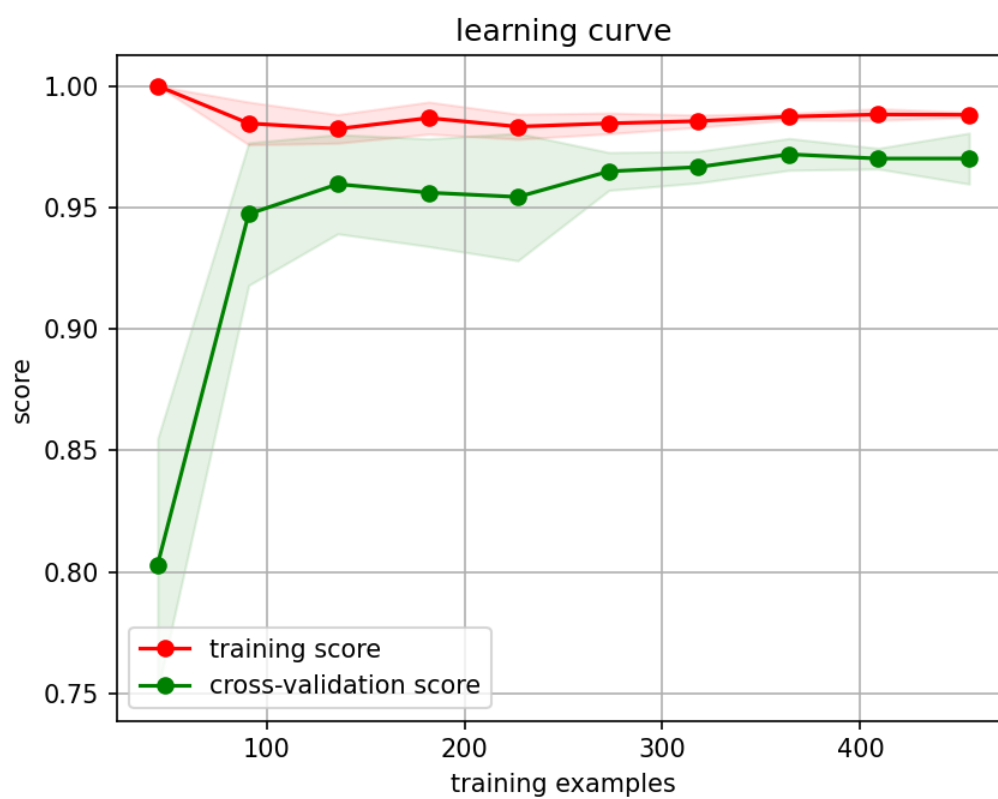
训练集准确率：0.9912

训练集混淆矩阵：

```
[[167 2]
 [2 284]]
```

### 3. 绘制学习曲线

- 使用 `learning_curve` 分析不同训练集大小对模型性能的影响。



## 扩展任务

### 4. 处理类别不平衡问题

- 在 SVC 中设置 `class_weight='balanced'`，对比平衡前后的模型性能。

平衡前：

测试集结果：

正则化参数 `c=0.1`：

测试集准确率：0.9737

测试集混淆矩阵：

```
[[41 2]
 [1 70]]
```

正则化参数 `c=1`：

测试集准确率：0.9561

测试集混淆矩阵：

```
[[41 2]
 [3 68]]
```

正则化参数 `c=10`：

测试集准确率：0.9649

测试集混淆矩阵：

```
[[42 1]
 [3 68]]
```

平衡后：

训练集结果：

正则化参数 `c=0.1`：

训练集准确率：0.9868

训练集混淆矩阵：

```
[[165 4]
 [2 284]]
```

正则化参数 `c=1`：

训练集准确率：0.9868

训练集混淆矩阵：

```
[[166 3]
 [3 283]]
```

正则化参数 `c=10`：

训练集准确率：0.9912

训练集混淆矩阵：

```
[[167 2]
 [2 284]]
```

## 5. 超参数调优

- 使用 GridSearchCV 搜索最优的 C 和 gamma 组合。

不同核函数性能比较：

rbf核函数结果：

测试集准确率：0.9649

测试集混淆矩阵：

```
[[41 2]
 [2 69]]
```

开始网格搜索...

网格搜索结果：

最优参数组合：{'C': 10, 'gamma': 0.01}

最优模型得分：0.9736263736263737

自定义核函数测试

## 6. 自定义核函数

- 实现一个简单的自定义核函数（如线性组合核）。

```
[2 69]]
```

多项式核函数结果：

测试集准确率：0.8684

测试集混淆矩阵：

```
[[28 15]
 [0 71]]
```

自定义核函数测试...

自定义核函数结果：

测试集准确率：0.9474

测试集混淆矩阵：

```
[[40 3]
 [3 68]]
```

# 五、实验分析

## 测试案例

```
python from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1]}
grid = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5)
grid.fit(X_train, y_train)
print("最优参数:", grid.best_params_)
```

## 预期结果

高斯核在非线性数据上表现优于线性核。

过大的  $C$  值可能导致过拟合（训练集准确率高，测试集低）。

## 六、思考题

1. SVM 为何对特征缩放敏感？如何解决这一问题？

原因：SVM 基于距离度量（如核函数、间隔计算），不同特征尺度会导致距离计算偏向大尺度特征

解决：标准化（如  $Z$ -score）或归一化特征，使所有特征具有相同量纲

2. 解释支持向量的几何意义及其对模型泛化能力的影响。

几何意义：距离决策边界最近的样本点，决定了边界的位置和方向

泛化影响：仅依赖支持向量，减少过拟合风险；支持向量越少模型越简单

3. 当数据维度远大于样本数量时，应选择何种核函数？为什么？

推荐选择：线性核（`kernel='linear'`）

原因：高维空间本身已具备线性可分性（Cover 定理），复杂核函数易导致过拟合

4. 分析软间隔 SVM 中参数  $C$  的物理意义。

控制分类错误的惩罚强度：

$C$  越大  $\rightarrow$  对误分类容忍度越低  $\rightarrow$  模型更复杂（可能过拟合）

$C$  越小  $\rightarrow$  允许更多误分类  $\rightarrow$  模型更简单（可能欠拟合）

## 七、参考文献

Scikit-learn 工具包指南：

<https://scikit-learn.org/stable/modules/svm.html>



**提示：**

调试时可使用交叉验证避免过拟合，例如：

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(SVC(kernel='linear'), X_train, y_train, cv=5)
print("交叉验证平均得分:", scores.mean())
```